Master-Slave Replication Engine

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a replicated key-value store that runs as three separate processes cooperating over TCP. At any given moment one of the three is the *master*: it accepts writes from clients, assigns each write a monotonically increasing sequence number, logs it to a durable write-ahead log, and streams it to the other two processes (the *followers*). The followers apply every write in the exact order the master chose, so that all three nodes converge on the same state. When a client reads, any of the three nodes can answer, because all three hold the same data (or will, very shortly, once the write has propagated).

The interesting part of the project is not the key-value store, which is deliberately simple. The interesting part is everything surrounding it: a binary WAL with log sequence numbers, a streaming protocol that lets a follower join the cluster at any point and catch up from whatever state it is in, a heartbeat-based failure detector that notices when the master has gone silent, a leader-election procedure that promotes one of the surviving followers to be the new master without allowing two masters to coexist, and a sync-vs-async commit knob that exposes the fundamental tradeoff between write latency and data safety. When you are finished you will have built, in miniature, the same mechanism that PostgreSQL streaming replication, MySQL semi-synchronous replication, and MongoDB replica sets all use under the hood.

This project is deliberately more involved than the others in Project 02. Replication, failure detection, and leader election each introduce distributed-systems concerns that simpler single-node projects do not have. Expect to spend significant time on testing the failure modes: a bug that only appears when a follower disconnects for exactly the wrong second is the characteristic kind of distributed-systems bug and is the point of the exercise. Your benchmark will include a chaos test that kills nodes at random moments and verifies the cluster recovers correctly.

You are not building a consensus algorithm in the style of Raft. This is single-master replication with a simple failover rule, not a general-purpose consensus protocol; the safety guarantees are weaker and the implementation is proportionally simpler. Concurrent writes, transactions, SQL, and sharding are all out of scope.

2 What the Final Product Looks Like

When your project is complete, a user can start three server processes on three separate machines (or three terminals on one machine, or three Docker containers, or three EC2 instances — Section 4 explains the deployment options), connect a client to any of them, and observe the cluster working. A typical startup and session looks like this.

```
# terminal 1
$ ./kvdb --node-id 1 --port 6001 --repl-port 7001 \
         --peers 2@host2:7002,3@host3:7003 --data ./data1
[node 1] starting up...
[node 1] WAL replay: 0 records
[node 1] no master announced yet; waiting for quorum
[node 1] peer 2 connected
[node 1] peer 3 connected
[node 1] quorum reached; starting election
[node 1] elected MASTER for term 1 (lsn=0, peers agreed: 2 of 2)
[node 1] accepting client writes on port 6001

# terminal 2
$ ./kvdb --node-id 2 --port 6002 --repl-port 7002 \
         --peers 1@host1:7001,3@host3:7003 --data ./data2
[node 2] starting up...
[node 2] WAL replay: 0 records
[node 2] connected to master at node 1 (term 1)
```

```
[node 2] streaming from lsn=0

# terminal 3
$ ./kvdb --node-id 3 --port 6003 --repl-port 7003 \
         --peers 1@host1:7001,2@host2:7002 --data ./data3
[node 3] starting up...
[node 3] connected to master at node 1 (term 1)
[node 3] streaming from lsn=0

# terminal 4 (client)
$ ./kvdb-cli host1 6001
connected to node 1 (role=MASTER, term=1)
> PUT alice 100
OK (lsn=1, replicated-to=2)
> PUT bob 50
OK (lsn=2, replicated-to=2)
> GET alice
100
> \info
node_id:  1
role:     MASTER
term:     1
lsn:      2
followers: node 2 (lag=0), node 3 (lag=0)
sync_mode: async
> QUIT

# verify the follower sees the same data
$ ./kvdb-cli host2 6002
connected to node 2 (role=FOLLOWER, term=1)
> GET alice
100
> GET bob
50
> \info
node_id:  2
role:     FOLLOWER
term:     1
lsn:      2
master:   node 1
lag:      0 ms
> QUIT
```

2.1 Failover Demonstration

Now the instructive part. Kill the master and watch the cluster elect a new one.

```
# terminal 1: the master
^C
[node 1] shutting down...

# terminal 2 (shortly after)
[node 2] master heartbeat missing (1500ms)
[node 2] master heartbeat missing (2000ms)
[node 2] master heartbeat missing (2500ms)
[node 2] master heartbeat missing (3000ms)
[node 2] master presumed dead; starting election for term 2
[node 2] vote request to node 1: FAILED (unreachable)
[node 2] vote request to node 3: GRANTED (my lsn=2 >= their lsn=2)
[node 2] received majority (2 of 3 including self)
[node 2] promoted to MASTER for term 2 (lsn=2)
[node 2] accepting client writes on port 6002

# terminal 3 (around the same time)
[node 3] master heartbeat missing (3000ms)
[node 3] received election from node 2 for term 2
```

```
[node 3] vote granted: node 2's lsn=2 >= my lsn=2
[node 3] new master: node 2 (term 2)
[node 3] streaming from lsn=2

# terminal 4 (client, now connecting to new master)
$ ./kvdb-cli host2 6002
connected to node 2 (role=MASTER, term=2)
> PUT charlie 75
OK (lsn=3, replicated-to=1)
> \info
node_id:  2
role:     MASTER
term:     2
lsn:      3
followers: node 3 (lag=0)
sync_mode: async
```

Notice the `term` number. The original master was master for term 1. After the election, node 2 is master for term 2. Term numbers are monotonically increasing and serve as the tiebreaker that prevents two masters from coexisting: if the old master were to come back to life and try to reassert itself, the followers would see a lower term and refuse to accept its writes. Term numbers are the one idea in this project borrowed from consensus algorithms, and they do most of the work of keeping the system safe.

If node 1 is now restarted, it will discover that term 2 exists (because node 2 or node 3 will tell it during the initial handshake), step down to follower, and catch up from LSN 2 to LSN 3. The cluster returns to three nodes.

3 Deployment Options

The server is deliberately designed so that nothing about the protocol or the code depends on how the nodes are deployed. What matters is that each node knows how to reach the others over TCP. Pick whichever of the options below matches the resources you have; all three are acceptable and all three are tested against the same reference implementation.

3.1 Three Processes on One Machine (simplest)

Open three terminals on the same computer. Run three copies of the server with different `-node-id`, `-port`, `-repl-port`, and `-data` arguments, with the peer list pointing at `localhost` for all of them. This is the recommended development setup and the setup.

```
./kvdb --node-id 1 --port 6001 --repl-port 7001 \
       --peers 2@localhost:7002,3@localhost:7003 --data ./data1 &
./kvdb --node-id 2 --port 6002 --repl-port 7002 \
       --peers 1@localhost:7001,3@localhost:7003 --data ./data2 &
./kvdb --node-id 3 --port 6003 --repl-port 7003 \
       --peers 1@localhost:7001,2@localhost:7002 --data ./data3 &
```

3.2 Docker Compose

If you are comfortable with Docker, a `docker-compose.yml` with three services running the same image is a clean way to run the cluster in isolated containers on one machine. Each container gets its own data volume, exposes its ports to the host, and resolves the other containers' hostnames via the Docker network's built-in DNS. An example for skeleton `docker-compose.yml`:

```
services:
  node1:
    build: .
    command: --node-id 1 --port 6001 --repl-port 7001
             --peers 2@node2:7002,3@node3:7003 --data /data
    ports: ["6001:6001"]
    volumes: [ "./data1:/data" ]

  node2:
```

```
    build: .
    command: --node-id 2 --port 6002 --repl-port 7002
             --peers 1@node1:7001,3@node3:7003 --data /data
    ports: ["6002:6002"]
    volumes: [ "./data2:/data" ]

  node3:
    build: .
    command: --node-id 3 --port 6003 --repl-port 7003
             --peers 1@node1:7001,2@node2:7002 --data /data
    ports: ["6003:6003"]
    volumes: [ "./data3:/data" ]
```

Docker Compose is convenient for the chaos tests in Phase 3 because `docker stop node1` is a clean way to kill a node without terminating your shell.

Their are other ways to use docker in the project as well, such as use it just for database handling, or running different OS inside em, just to experiment project working as it's working on three different devices simultaneously.

3.3 Three EC2 Instances (or Any Three VMs)

Launch three small EC2 instances (`t3.micro` is sufficient), make sure the security group allows inbound TCP on the ports you will use (the replication ports especially), install your build dependencies, copy the binary to each instance, and start it with the peer list pointing at the public or private IP addresses of the other two instances. Test with `telnet` or `nc` before running the real server that each instance can reach the others on the replication ports. This option is the most realistic but the most tedious to set up; only choose it if you already have AWS credentials and want the practice.

3.4 Required Assumption: the Cluster Is Always Three Nodes

No matter which deployment option you choose, the cluster size is fixed at three nodes. Three is the smallest cluster size at which a meaningful majority quorum exists (two out of three), and it is what the failover algorithm in Phase 3 assumes. You do not need to support dynamic cluster membership, adding nodes at runtime, or clusters of a different size.

4 The Client-Facing Protocol

Clients speak to any node over its client port (`-port`). The protocol is plain text, one command per line, same format as the other Project 02 manuals.

- `PUT key value` — Store a key-value pair. Only accepted by a master. A follower receiving this command responds with an error naming the current master (so the client can reconnect).
- `DELETE key` — Delete a key. Same restriction.
- `GET key` — Return the value for a key, or `NULL`. Accepted by any node. Note that a follower may be slightly behind the master; the read reflects whatever state the follower has applied so far.
- `\info` — Print the node's current role, term, LSN, master identity, and (if this is the master) the list of followers with their replication lag.
- `\sync on` / `\sync off` — Switch the master between synchronous and asynchronous commit modes (Section 6.4). Only meaningful on the master.
- `QUIT` — Close the client connection.

Keys and values are strings of printable ASCII, no whitespace.

5 Architectural Overview

Each server is one process containing three components that communicate through shared state (protected by a mutex):

```
+----------------------+       +----------------------+
|  Client listener     |       |  Peer listener       |
|  port = --port       |       |  port = --repl-port  |
|  (one thread per     |       |  (one thread per     |
```

```
|   client connection) |        |   peer connection)   |
+----------+-----------+        +----------+-----------+
           |                               |
           v                               v
       +---+-------------------------------+---+
       |             Shared state              |
       |  - in-memory KV store                 |
       |  - WAL writer (append-only file)      |
       |  - role: MASTER / FOLLOWER            |
       |  - current term                       |
       |  - current LSN                        |
       |  - peer table                         |
       |  - master id (if FOLLOWER)            |
       +---------------------------------------+
                         ^
                         |
              +---------+---------+
              |   Background      |
              |   threads:        |
              |   - heartbeats    |
              |   - failover      |
              |   - reconnect     |
              +-------------------+
```

Keep the three network-facing components (client listener, peer listener, and outgoing peer connector) strictly separated from the shared state via a well-defined API. The shared state is the single source of truth and all mutations go through it under the lock.

6 The Write-Ahead Log

6.1 Format

The WAL is a single append-only binary file per node, living inside the `-data` directory. Each record has a 24-byte header followed by a variable-length body:

```
Offset  Size   Field           Description
------  -----  --------------  -------------------------------------
  0      4     crc32           CRC32 of bytes from offset 4 to end
  4      8     lsn             uint64, monotonically increasing
 12      8     term            uint64, the term in which this was
                                written (Section 7)
 20      1     op_type         1=PUT, 2=DELETE
 21      2     key_len         uint16
 23      2     value_len       uint16 (0 for DELETE)
 25      ...   key bytes       key_len bytes, no null terminator
 ...     ...   value bytes     value_len bytes, no null terminator
```

All integers are little-endian. LSNs start at 1 and are assigned by the master in the order that writes are accepted; LSN 0 means "no writes yet." The term field records which master produced the record. Terms are also monotonically increasing, and every new master increments the term when it is elected.

6.2 Writing and Fsync

The master writes to its WAL before applying the write to the in-memory KV store and before acknowledging the client. The order is: assign the next LSN, serialize the record, append it to the WAL file, call `fsync`, apply to the KV store, respond to the client.

The `fsync` is essential. Without it, a crash of the master after acknowledgement but before the kernel flushed the buffer would lose the acknowledged write, which violates durability. It is the same rule you saw in the WAL project (Project 02 Option 10) and for the same reason.

Followers also maintain their own WAL and `fsync` in the same way. When a follower receives a replicated record from the master, it writes it to its own WAL, `fsyncs`, applies it to its own KV store, and then sends an acknowledgement (an `ACK` message containing the LSN) back to the master.

6.3 WAL Replay on Startup

On startup, every node reads its own WAL from the beginning, verifies each record's CRC32, reconstructs the in-memory KV store, and records the highest LSN and term it has seen. These become the node's starting LSN and term. A corrupt record (CRC mismatch) or a truncated record (short read) at the end of the file indicates a crash mid-write; everything up to that point is kept, everything after is discarded.

6.4 Sync vs Async Modes

The master supports two commit modes, switchable at runtime via the `\sync` command:

- **Asynchronous (default)**: The master acknowledges the client as soon as its own WAL `fsync` completes and the write has been applied locally. Replication to followers happens in the background. A crash of the master between ack and follower replication can lose the acknowledged write if a newly elected master does not have it.
- **Synchronous**: The master waits for at least one follower's `ACK` before acknowledging the client. This guarantees that the write has durably survived on at least two nodes. The write latency is approximately the round-trip time to the fastest follower, so synchronous writes are slower.

The mode affects only the master's acknowledgement timing. In both modes, writes are streamed to followers as soon as the master writes them locally.

7 Terms and Leader Election

The single most important idea in this project is the *term number*. A term is a period of time during which at most one node is master. Terms are integers starting at 1 and increasing monotonically. Every WAL record, every replication message, and every election message carries the term of the master that initiated it.

7.1 The Core Rules

1. At any moment, every node remembers a "current term." On startup it is loaded from the WAL (the highest term seen in any replayed record). In a fresh cluster with empty WALs, every node starts at term 0.
2. A node never accepts a message from an earlier term. If a node currently at term 5 receives a heartbeat or a WAL record carrying term 4, it rejects the message and tells the sender about term 5. The sender then either steps down (if it thought it was master) or updates its own term.
3. Conversely, a node that sees a higher term than its own always updates. A term-7 message arriving at a node currently at term 5 makes the node immediately adopt term 7, step down to follower if it was master, and accept the message if otherwise valid.
4. When a follower believes the master has died (Section 7.3), it tries to become master for the next term. Only if it gets votes from a strict majority of the cluster (two out of three) does it succeed.

These four rules, together, prevent split-brain: there is at most one master per term, no two elections can both win, and if the old master revives it will see the new term and step down.

7.2 Heartbeats

The master sends a heartbeat message to each follower every 500 milliseconds on the replication connection, in addition to any WAL records that happen to be in flight. A heartbeat is a tiny message carrying only the master's term and current LSN.

A follower tracks the time of the most recent heartbeat (or WAL record, which counts as an implicit heartbeat) from its master. If that time exceeds 3,000 milliseconds, the follower declares the master dead and begins an election. The 3-second timeout is deliberately long compared to the heartbeat interval so that a brief pause — a garbage collection, a momentary network hiccup — does not trigger a false failover.

7.3 Elections

When a follower decides to run for master, it does the following, atomically with respect to the shared state:

1. Increment the current term locally.
2. Set its role to `CANDIDATE`.
3. Vote for itself.
4. Send a `VOTE_REQUEST` message to every other node in the peer list, containing its node id, its candidate term, and its own current LSN.

A node that receives a `VOTE_REQUEST` applies these rules, in order:

1. If the request's term is less than the recipient's current term, reject the vote.
2. If the recipient has already voted in this term (for itself or for someone else), reject the vote.
3. If the requester's LSN is less than the recipient's LSN, reject the vote. This is the safety rule that prevents a follower with stale data from becoming master. Only a follower with at least as much data as the recipient has should be allowed to lead.
4. Otherwise, grant the vote, record the recipient's choice for this term (so rule 2 works on any subsequent request), and reply.

The candidate counts votes, including its own. If it receives a strict majority (two out of three), it transitions to `MASTER`, writes nothing to its WAL at this moment (the first thing it will write under the new term is the first client command it accepts), and begins sending heartbeats. If it does not receive a majority within 1 second of starting the election, it times out and tries again — incrementing the term a second time, resetting its vote, and sending a new round of `VOTE_REQUEST`s. The retry loop has a small random delay before each attempt to avoid repeated ties (two nodes racing to become master can deadlock otherwise).

7.4 The Split-Brain Guarantee, Informally

Why can there not be two masters for the same term? Because becoming master for a term requires votes from at least two of the three nodes. Two disjoint majorities of size 2 cannot exist in a cluster of size 3 (any two such sets would share at least one node, which cannot vote for two different candidates in the same term by rule 2). Therefore at most one candidate wins any given term, and therefore at most one master exists at any given time. This is the same argument that makes Raft safe; the only difference is that Raft goes on to use the log replication protocol to guarantee even stronger properties, while this project accepts a weaker one.

8 Phase 1 — Single-Node KV Store with WAL

The goal of Phase 1 is a standalone key-value store with a durable write-ahead log. No replication, no peers, no terms, no failover. A single process accepts client connections, supports `PUT`/`GET`/`DELETE`, writes to its WAL, and recovers correctly on restart.

Build these components:

- A TCP server that accepts multiple concurrent clients. One thread per client is the simplest approach.
- The in-memory KV store: a hash map from string keys to string values, protected by a mutex.
- The WAL writer: binary file format from Section 6.1, `fsync` after every record, LSN assignment.
- WAL replay on startup: read from the beginning, verify CRCs, rebuild the in-memory store, record the highest LSN seen.
- A command-line client (`kvdb-cli`) that connects to a server, reads commands from stdin, sends them, and prints responses.
- The `\info` command on the server side, reporting role (always `MASTER` for now), term (0), and current LSN.

At the end of Phase 1, the server should handle many clients simultaneously, persist writes across restarts, and correctly reject corrupt records if the WAL has been truncated or damaged.

9 Phase 2 — Two-Node Asynchronous Replication

The goal of Phase 2 is to add replication between a master and a single follower, running asynchronously. At the end of Phase 2 you should be able to start two nodes, write to the master, and observe the follower converging. Three-node support, heartbeats, and failover are not yet required — they come in Phase 3.

9.1 Peer Listener and Connector

Add a second TCP port (`-repl-port`) that accepts connections from follower nodes. On the follower side, add code that connects out to the master's replication port as soon as the follower starts.

The peer connection is full-duplex: the master sends WAL records and heartbeats; the follower sends acknowledgements. Use two threads per connection (one for each direction) or a single thread with non-blocking reads and writes, your choice.

9.2 The Replication Handshake

When a follower connects to a master, the first thing it sends is a `HELLO` message containing its `node_id`, its current term, and its current LSN. The master replies with its own `node_id` and term, and then:

- If the follower's term is greater than the master's, the master rejects the connection (the follower has seen a newer master somewhere; something is confused). This will not happen in Phase 2 but will matter in Phase 3.
- If the follower's term is less, the master proceeds.
- If the follower's LSN is greater than the master's, reject the connection (the follower somehow has data the master does not). This should never happen in correct operation.
- Otherwise, the master begins streaming WAL records to the follower starting from the follower's LSN + 1.

The master must serve both the client-facing port and the replication port concurrently and correctly; a slow client must not block replication, and a slow follower must not block clients.

9.3 Replication Stream Format

Each WAL record sent over the replication connection uses the same 24-byte header and body as the on-disk format (Section 6.1). This means the master can often literally copy bytes from its WAL file to the socket without reserializing them, which is efficient.

After sending each record, the master continues to stream new records as they are written by client activity. The stream never closes normally; it only closes when one side disconnects.

9.4 Follower Behavior

When a follower receives a WAL record on the replication stream:

1. Verify the CRC32.
2. Check that the LSN is exactly one greater than the follower's current LSN. A gap indicates a protocol violation; close the connection and reconnect.
3. Append the record to the follower's own WAL.
4. `fsync` the WAL.
5. Apply the record to the follower's in-memory KV store.
6. Update the follower's current LSN.
7. Send an `ACK` message back to the master containing the new LSN.

9.5 Reconnection and Catch-up

If the replication connection drops (for any reason: network hiccup, master crash, socket error), the follower must reconnect and resume. Use exponential backoff: wait 1 second, then 2, then 4, capping at 10 seconds. On each reconnection attempt, redo the handshake. Because the handshake sends the follower's current LSN, catch-up happens automatically: the master starts streaming from wherever the follower left off.

A follower that starts with an empty WAL simply sends LSN 0 in the handshake, and the master streams everything from the beginning. The catch-up path and the normal streaming path are the same code.

9.6 `PUT` Flow Under Async Replication

When the master receives a `PUT` from a client:

1. Under the shared-state lock, assign the next LSN, build the WAL record.
2. Append to the master's WAL and `fsync`.

3. Apply to the master's KV store.
4. Respond `OK` to the client immediately, reporting the LSN.
5. Hand the record off to the replication streamer, which queues it for sending to every connected follower.

The client's response does not wait for any follower. This is the asynchronous mode.

10 Phase 3 — Three Nodes, Sync Mode, and Failover

The goal of Phase 3 is to bring the system up to a real three-node cluster that tolerates a single failure. This phase adds heartbeats, elections, term numbers, and synchronous commit.

10.1 Three-Node Startup

The server now accepts a peer list with two entries (`-peers 2@host:port,3@host:port`) and needs to establish replication connections with both peers. At startup, neither node is automatically the master. Instead:

1. Each node starts up, replays its WAL, and enters a `WAITING` state with its current term from the WAL (or 0 in a fresh cluster).
2. Each node attempts to connect to each peer. When connected, the nodes exchange `HELLO` messages.
3. Once a node knows the term of at least one other peer, it adopts the highest term it has seen.
4. After a short startup delay (e.g., 2 seconds) or as soon as the node has heard from both peers, if no master has been observed in the current term, the node starts an election using the procedure in Section 7.3.
5. When an election succeeds, the elected node announces itself as master, and the others become followers.

Existing clusters (nodes with non-empty WALs and term > 0) skip the election if they already hear from a master in their current term; they simply become followers and resume streaming. A cold start with empty WALs begins at term 0; the first successful election advances the cluster to term 1.

10.2 Heartbeats

Add a background thread on the master that, every 500 ms, sends a `HEARTBEAT` message to each follower carrying the master's current term and LSN. The heartbeat serves two purposes: it keeps the follower's timeout from expiring, and it lets the follower detect if its own LSN has fallen behind (useful diagnostic information for the `\info` command).

On the follower side, add a thread that wakes every 500 ms and checks the time since the last heartbeat (or WAL record) from the current master. If more than 3,000 ms have passed, the master is presumed dead and the follower initiates an election.

10.3 Elections

Implement `VOTE_REQUEST` and `VOTE_RESPONSE` messages according to Section 7.3. Elections must be atomic with respect to the shared state: two followers racing to become master must both see the increment of the term. Use the shared state lock.

Once a candidate has won, it transitions to master and begins sending heartbeats. Existing followers see the new heartbeats (with the new term) and update their current term and master identity accordingly. If a node is in the middle of an election when it receives a heartbeat from a new master with a term greater than its own, it abandons its candidacy and becomes a follower of the new master.

10.4 Rejoin of a Former Master

When a crashed master is restarted, it replays its WAL, loads its old term, and attempts to connect to the peers. During the handshake it discovers a higher term than its own, updates to the new term, and becomes a follower. Its WAL may contain records from its old term that were never replicated (for example, if it crashed after appending to its own WAL but before any follower acknowledged). These records are at LSNs greater than the cluster's current LSN.

For this project, the simplest correct handling is: on becoming a follower with a newer term, truncate the local WAL to the cluster's current LSN (which the follower can learn from the master's first heartbeat)

and discard any in-memory state past that point. This loses the un-replicated writes from the old term, which is correct: those writes were never acknowledged to a majority, so clients were never told they committed (assuming sync mode) or were told they committed but the cluster is allowed to lose them (async mode, as the manual for async explicitly warned).

10.5 Sync Mode

Modify the `PUT` flow so that when sync mode is enabled, the master does not respond `OK` to the client until it has received an `ACK` from at least one follower for the written LSN. Implement this with a per-LSN condition variable or a simple map from LSN to a pending-clients list. Each follower `ACK` wakes the clients waiting on that LSN (if their LSN is covered by the `ACK`), and those clients then return to their callers.

10.6 Chaos Testing

Write a chaos test program (a shell script is fine) that runs the three-node cluster and performs the following sequence, checking invariants between steps:

1. Start all three nodes. Wait for a master to be elected. Write 1000 key-value pairs through the master. Confirm that all three nodes have the same data (by connecting to each one and running a checksum query, or by reading the WAL files and comparing).
2. Kill the master. Wait for failover. Confirm that a new master has been elected and that it has LSN 1000. Write 1000 more key-value pairs through the new master.
3. Restart the killed node. Confirm that it rejoins as a follower and catches up to LSN 2000.
4. Kill a follower (not the master this time). Write 500 more key-value pairs. Confirm that the remaining two nodes (master and one follower) continue to accept writes.
5. Restart the killed follower. Confirm that it catches up to LSN 2500.
6. In sync mode, kill two nodes simultaneously. Confirm that the remaining master stops accepting writes (no majority possible for the `ACK`). Restart one of the killed nodes. Confirm that writes resume.

The chaos test is the single most important deliverable of Phase 3. A submission that works in the happy path but fails one of these scenarios will receive a reduced grade for Phase 3 proportional to the number of scenarios that fail.

11 Deliverables and Submission

11.1 What to Submit

A single zip archive containing:

- The complete source tree of your implementation, including both the server (`kvdb`) and the client (`kvdb-cli`).
- A `Makefile` (or `CMakeLists.txt`) that builds both binaries with a single command on a recent Ubuntu LTS.
- A `README.md` explaining how to build and run the project, how to start a three-node cluster in each of the three deployment options from Section 4, the list of supported client commands, and any known limitations.
- A `design.pdf` document describing: the server architecture and the separation between the client listener, peer listener, and shared state; the WAL format (essentially repeating Section 6.1 in your own words); the replication protocol, including the handshake, streaming, and `ACK` messages; the term-based safety rules and the election algorithm; the sync mode implementation; and a discussion of each of the six chaos test scenarios, including the observed behavior in your implementation.
- A `chaos/` directory with the chaos test script, a `README` describing each scenario it runs, and logs from your final run showing each scenario passing.
- A `docker-compose.yml` or equivalent for whichever deployment option you tested with, so the grader can reproduce your setup.

11.2 Archive Naming

`Group[Number]_Project02_Replication.zip`, for example `Group17_Project02_Replication.zip`.

11.3 Demonstration

Each group will present a live demonstration during the week following the submission evaluation. The demonstration will be longer than for other projects because of the number of scenarios that need to be exercised. It consists of three parts. First, a walkthrough of the architecture using the design document. Second, a live startup of the three-node cluster followed by the full chaos test script with the instructor free to propose additional scenarios on the spot ("what happens if I kill the master exactly while a client is writing in sync mode?"). Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component of the implementation, with particular focus on the term-based safety rules and why they prevent split-brain. All group members must be present. Missing the demonstration results in a zero for Phase 3.

12 Bonus Opportunities

Each of the following is independent and may be attempted in any combination. Bonus credit is awarded on top of the base grade up to a maximum of 20 percentage points total (higher than other projects because of the increased difficulty of the base). To claim bonus credit, your design document must include a dedicated section explaining which bonus items you attempted and how you verified them.

12.1 Read-Your-Writes Consistency (up to 5%)

Currently, a client that writes to the master and then reads from a follower may see stale data if the follower has not yet caught up. Implement a consistency guarantee: when a client writes and receives `OK (lsn=N)`, any subsequent read through the client library from *any* node waits (if necessary) until that node has applied LSN N before returning. The client library will need to track the highest LSN it has seen in any response and include it in every subsequent request.

12.2 Snapshots and Log Compaction (up to 5%)

The WAL grows without bound in the base project. Implement a snapshot mechanism: periodically (manually via a `\snapshot` command is sufficient), the node writes its entire in-memory KV store to a snapshot file and truncates its WAL to only the records after the snapshot. On startup, a node first loads the most recent snapshot, then replays the WAL records after it. Follower catch-up from scratch uses the snapshot as a starting point (the master sends the snapshot file, then streams the WAL from there), which can be much faster than replaying the entire history.

12.3 Five-Node Cluster (up to 10%)

Extend the project to support a five-node cluster with a majority of three. This is a significant change to testing because the set of failure scenarios grows: the cluster now tolerates two simultaneous node failures, elections have five participants, and the chaos tests must be extended to cover the additional scenarios. Your design document must explain which parts of the code were hardcoded to three nodes and how you generalized them.

13 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing replication library, consensus library, or distributed coordination service. This includes but is not limited to etcd, ZooKeeper, Consul, `libraft`, `canonical/raft`, `hashicorp/raft`, `logcabin`, and their language bindings.
- Use of any existing database or storage engine as a component (PostgreSQL streaming replication, MySQL replication, Redis replication, LevelDB, RocksDB, SQLite, etc.).
- Use of any existing binary serialization or RPC library (Protobuf, Cap'n Proto, FlatBuffers, gRPC, Thrift). The wire format is specified in Section 6 and must be implemented with direct reads and writes on a plain TCP socket.
- Use of Python at any layer of the system, including test harnesses and the chaos test driver. Tests and scripts must be written in C, C++, shell, or JavaScript.
- Skipping `fsync` on WAL writes and calling the result durable. Same rule as Project 02 Option 10: a write is only durable after `fsync` returns.
- Allowing two masters to coexist, even briefly. If your cluster can ever be in a state where two nodes believe they are master for the same term, the failover logic is broken regardless of whether

it is easy to trigger; fix the bug. The chaos test must verify this invariant.

- Submitting code whose commit history shows large, infrequent commits. Your git history should reflect the incremental development of a system of this size; histories with three commits totaling thousands of lines will be scrutinized for academic integrity.

14 Required Reading

The first two items are required reading before you start Phases 2 and 3 respectively. The remaining items are reference material.

1. PostgreSQL documentation, chapter "High Availability, Load Balancing, and Replication," specifically Section 27.2 on Log-Shipping Standby Servers and Section 27.2.5 on Streaming Replication. This is the most direct description of the kind of replication you are building, written from the point of view of the system that invented streaming replication in its modern form. Read before Phase 2.
2. Ongaro, D. and Ousterhout, J. "In Search of an Understandable Consensus Algorithm." *Proceedings of USENIX Annual Technical Conference*, 2014. The Raft paper. You are not implementing Raft, but the term-based safety argument in Section 5.4 is exactly the argument underlying your leader-election rules in Section 7 of this manual. Read Sections 5.1 through 5.4 before Phase 3.
3. Kleppmann, M. *Designing Data-Intensive Applications*, Chapter 5 ("Replication"). A clear, high-level discussion of single-leader replication, the sync/async tradeoff, failover, and the problems that can go wrong. Read it as background; it will help you understand why the algorithm is the way it is.
4. "The Secret Lives of Data: Raft," an interactive animation at `http://thesecretlivesofdata.com/raft/`. Optional but highly recommended; working through the animation for a few minutes will give you a much clearer intuition for term numbers and election behavior than any written description.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM